Module 2: Data Link Layer

Contents:*Data Link Layer: Data Link Layer Services, Error Detection and Correction: Introduction, Error Detection, Error Correction. Linear Block Codes: hamming code, Hamming Distance, parity check code. Cyclic Codes: CRC (Polynomials), Advantages of Cyclic Codes, Other Cyclic Codes (Examples: CHECKSUM: One's Complement, Internet Checksum). Framing: fixed-size framing, variable size framing. Flow control: flow control protocols. Noiseless channels: simplest protocol, stop-and-wait protocol. . Noisy channels: stop-and-wait Automatic Repeat Request (ARQ), go-back-n ARQ, Selective repeat ARQ, piggybacking*

Unit Objectives and outcomes: *On completion the students will be able to:*

1.To understand services and protocols used at Physical, Data Link, Network, Transport Layer.

Unit outcomes **:** On completion the students will be able to

CO2: Analyze data link layer services, error detection and correction, linear block codes, cyclic codes, framing and flow control protocols.

Outcome Mapping:

PEOs: POs: COs: 2 PSOs:

Books used

1.Simon Haykin ," Communication Systems" , 3E
2.Ranjan Bose, —Information Theory coding and Cryptography‖, McGraw-Hill, 2nd Ed
3.Bernad Sklar, —Digital Communication Fundamentals & applications‖, Pearson Education. Second Edition.

Unit-II: Data Link Layer Part- I

Introduction:-

Data Link Layer is second layer of OSI Layered Model. The communication channel that connects the adjacent nodes is known as links, and in order to move the datagram from source to the destination, the datagram must be moved across an individual link.

The main responsibility of the Data Link Layer is to transfer the datagram across an individual link.

The Data link layer protocol defines the format of the packet exchanged across the nodes as well as the actions such as Error detection, retransmission, flow control, and random access.

An important characteristic of a Data Link Layer is that datagram can be handled by different link layer protocols on different links in a path. For example, the datagram is handled by Ethernet on the first link, PPP on the second link.

Data link layer has two sub-layers:

Logical Link Control: It deals with protocols, flow-control, and error control

Media Access Control: It deals with actual control of media

Functionality of Data-link Layer

- **Framing**
- **Addressing**
- **Synchronization**
- **Error Control**
- **Flow Control**
- **Multi-Access**

Data Link Layer Services

The data link layer offers three types of services.

Unacknowledged connectionless service − Here, the data link layer of the sending machine sends independent frames to the data link layer of the receiving machine.

Acknowledged connectionless service − Here, no logical connection is set up between the host machines, but each frame sent by the source machine is acknowledged by the destination machine on receiving.

Acknowledged connection-oriented service − This is the best service that the data link layer can offer to the network layer.

Set up of connection – A logical path is set up between the source and the destination machines.

Sending frames – The frames are transmitted.

Release connection – The connection is released, buffers and other resources are released.

Error Detection and Correction

Introduction

•There are many reasons such as noise, cross-talk etc., which may help data to get corrupted during transmission.

•The upper layers work on some generalized view of network architecture and are not aware of actual hardware data processing. Hence, the upper layers expect error-free transmission between the systems.

•Most of the applications would not function expectedly if they receive erroneous data.

•Applications such as voice and video may not be that affected and with some errors they may still function well.

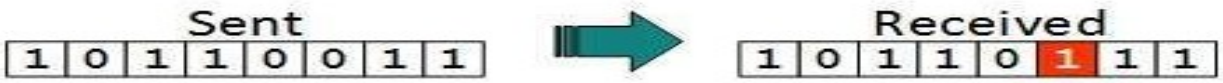
•Data-link layer uses some error control mechanism to ensure that frames (data bit streams) are transmitted with certain level of accuracy.

To understand how errors is controlled, it is essential to know what types of errors may occur.

Types of Errors

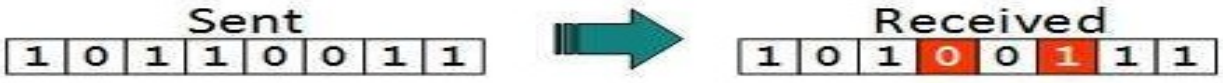
There may be three types of errors:

- **Single bit error**



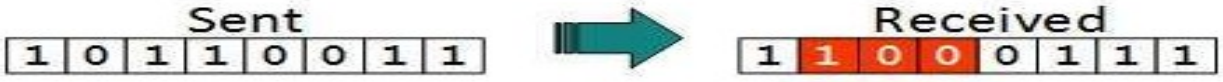
In a frame, there is only one bit, anywhere though, which is corrupt.

- **Multiple bits error**



Frame is received with more than one bits in corrupted state.

- **Burst error**



Frame contains more than1 consecutive bits corrupted.

Error control mechanism may involve two possible ways:

Error Detection

- Error correction involves ascertaining the exact number of bits that has been corrupted and the location of the corrupted bits.
- For both error detection and error correction, the sender needs to send some additional bits along with the data bits.
- The receiver performs necessary checks based upon the additional redundant bits. If it finds that the data is free from errors, it removes the redundant bits before passing the message to the upper layers.
- Errors in the received frames are detected by means of Parity Check and Cyclic Redundancy Check (CRC).

Error Detection Techniques

Parity Check

- A parity bit is a check bit, which is added to a block of data for error detection purposes.
- The value of the parity bit is assigned either 0 or 1 that makes the number of 1s in the message block either even or odd depending upon the type of parity.
- Parity check is suitable for single bit error detection only.

The two types of parity checking are

- **Even Parity** − Here the total number of bits in the message is made even.

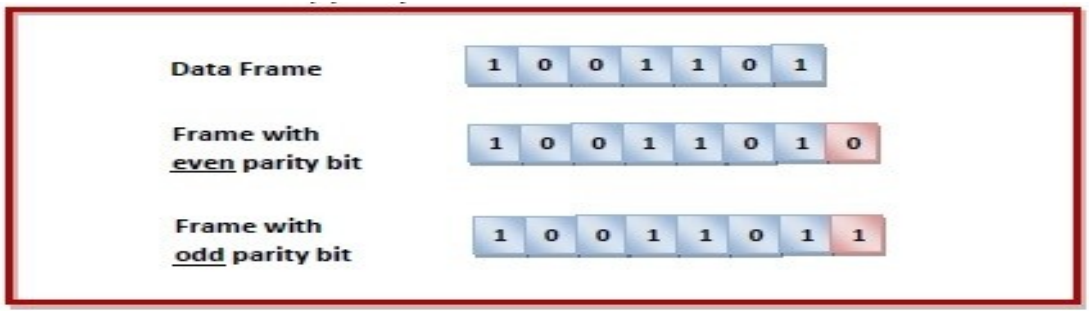
- **Odd Parity** − Here the total number of bits in the message is made odd.

Error Detection by Adding Parity Bit

Sender's End − While creating a frame, the sender counts the number of 1s in it and adds the parity bit in following way

In case of even parity − If number of 1s is even, parity bit value is 0. If number of 1s is odd, parity bit value is 1.

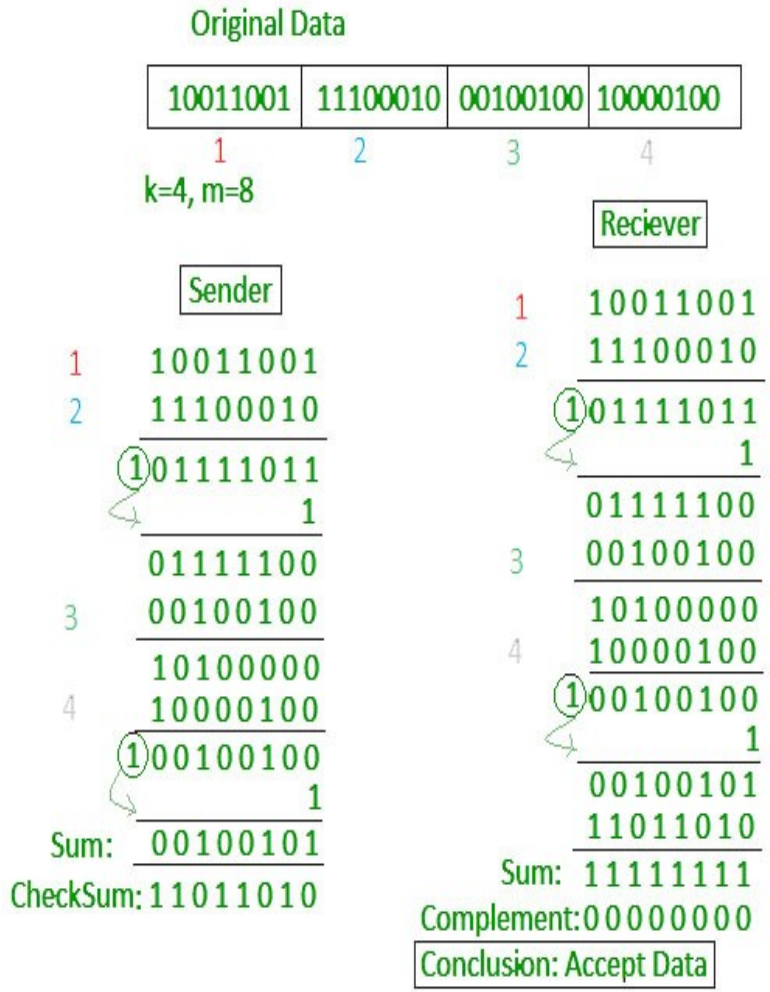
In case of odd parity − If number of 1s is odd, parity bit value is 0. If number of 1s is even, parity bit value is 1.



Receiver's End − On receiving a frame, the receiver counts the number of 1s in it.
In case of even parity check, if the count of 1s is even, the frame is accepted, otherwise it is rejected.
In case of odd parity check, if the count of 1s is odd, the frame is accepted, otherwise it is rejected.

Checksum (1s Complement)

- In the sender's end the segments are added using 1's complement arithmetic to get the sum. The sum is complemented to get the checksum.
- The checksum segment is sent along with the data segments.
- In checksum error detection scheme, the data is divided into k segments each of m bits.
- At the receiver's end, all received segments are added using 1's complement arithmetic to get the sum. The sum is complemented.
- If the result is zero, the received data is accepted; otherwise discarded.



Cyclic Codes: Cyclic Redundancy Check (CRC)

- CRC is a different approach to detect if the received frame contains valid data.
- This technique involves binary division of the data bits being sent. The divisor is generated using polynomials.

CRC uses Generator Polynomial which is available on both sender and receiver side.

An example generator polynomial is of the form like $x^3 + x + 1$. This generator polynomial represents key 1011.

n : Number of bits in data to be sent from sender side.

k : Number of bits in the key obtained from generator polynomial.

Sender Side (Generation of Encoded Data from Data and Generator Polynomial (or Key)):

1.The binary data is first augmented by adding k-1 zeros in the end of the data

2.Use ***modulo-2 binary division*** to divide binary data by the key and store remainder of division.

3.Append the remainder at the end of the data to form the encoded data and send the same.

Receiver Side (Check if there are errors introduced in transmission)

1.Perform modulo-2 division again and if the remainder is 0, then there are no errors.

Modulo 2 Division:

1.The process of modulo-2 binary division is the same as the familiar division process we use for decimal numbers. Just that instead of subtraction, we use XOR here.

2.In each step, a copy of the divisor (or data) is XORed with the k bits of the dividend (or key).

3.The result of the XOR operation (remainder) is (n-1) bits, which is used for the next step after 1 extra bit is pulled down to make it n bits long.

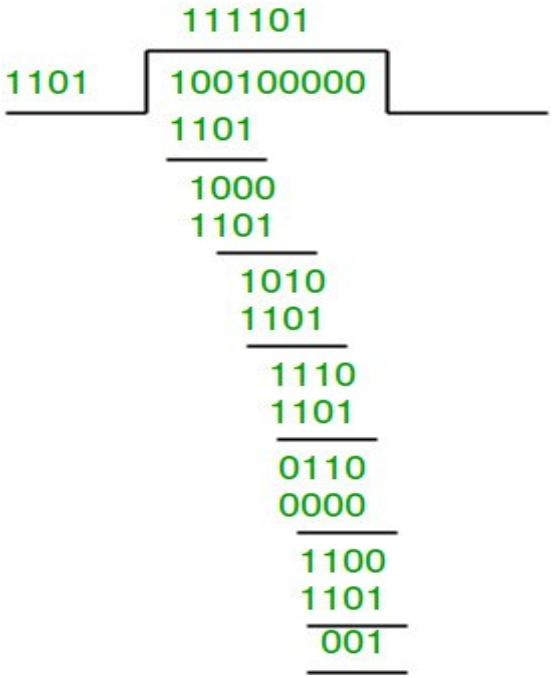
4.When there are no bits left to pull down, we have a result. The (n-1)-bit remainder which is appended at the sender side.

Example 1 (No error in transmission):

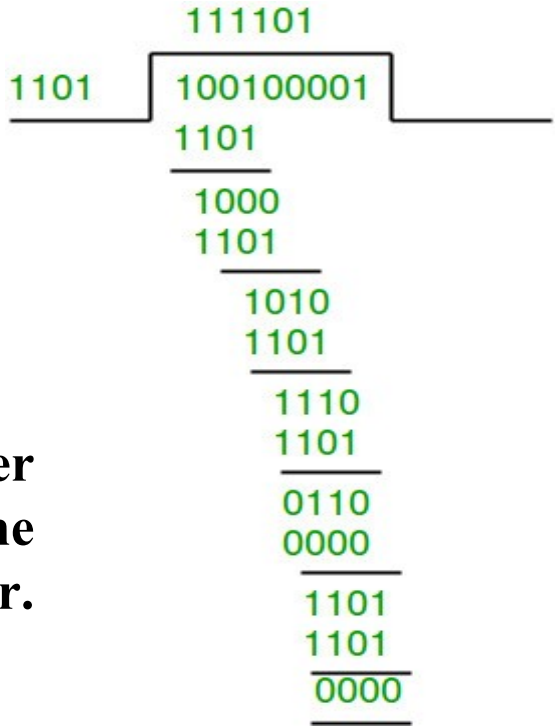
Data word to be sent - 100100 Key - 1101 [Or generator polynomial $x^3 + x^2 + 1$

Sender side

Therefore, the remainder is 001 and hence the encoded data sent is 100100001.



Receiver Side: Code word received at the receiver side 100100001



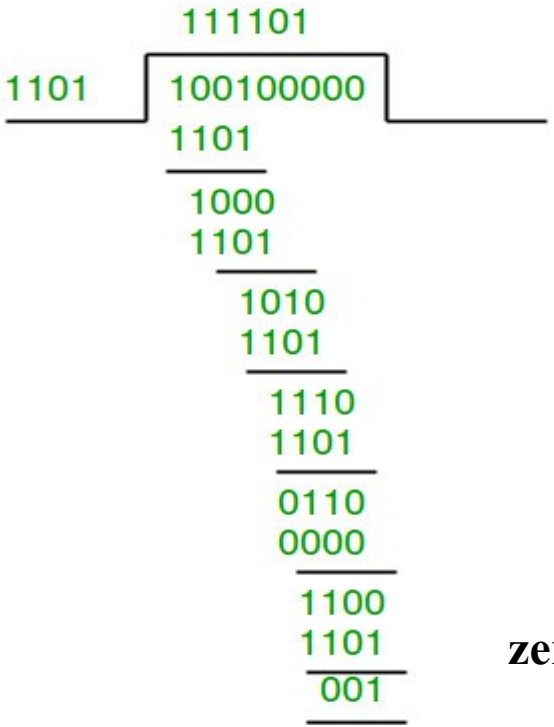
Therefore, the remainder is all zeros. Hence, the data received has no error.

Example 2: (Error in transmission)

Data word to be sent - 100100 Key - 1101

Sender Side:

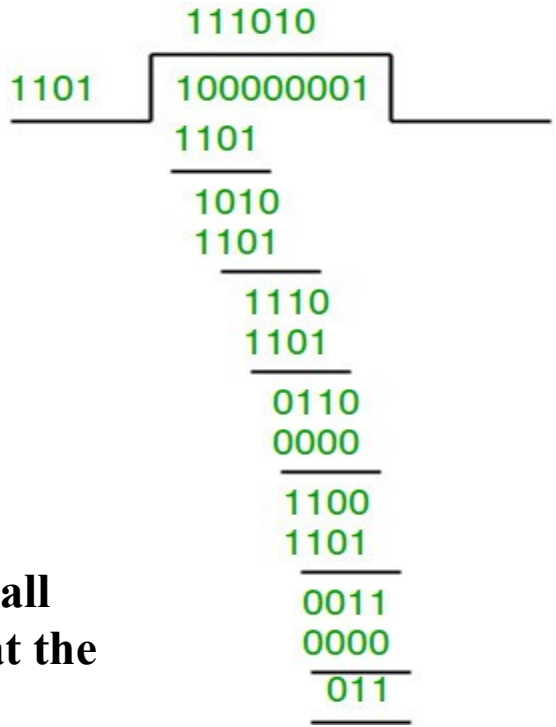
Therefore, the remainder is 001 and hence the code word sent is 100100001.



Receiver Side

Let there be an error in transmission media

Code word received at the receiver side - 100000001



Since the remainder is not all zeroes, the error is detected at the receiver side.

Advantages of Cyclic Codes

- The cyclic codes have a very good performance in detecting single-bit errors, double errors, an odd number of errors, and burst errors.
- They can easily be implemented in hardware and software.
- They are especially fast when implemented in hardware.
- This has made cyclic codes a good candidate for many networks.

Error Correction

Error-correcting codes (ECC) are a sequence of numbers generated by specific algorithms for detecting and removing errors in data that has been transmitted over noisy channels.
Error correcting codes ascertain the exact number of bits that has been corrupted and the location of the corrupted bits, within the limitations in algorithm.

ECCs can be broadly categorized into two types −

Block codes − The message is divided into fixed-sized blocks of bits, to which redundant bits are added for error detection or correction.

Convolutional codes − The message comprises of data streams of arbitrary length and parity symbols are generated by the sliding application of a Boolean function to the data stream.

Hamming Code

- Hamming code is a block code that is capable of detecting up to two simultaneous bit errors and correcting single-bit errors.
- It was developed by R.W. Hamming for error correction.
- In this coding method, the source encodes the message by inserting redundant bits within the message.
- These redundant bits are extra bits that are generated and inserted at specific positions in the message itself to enable error detection and correction.
- When the destination receives this message, it performs recalculations to detect errors and find the bit position that has error.

Encoding a message by Hamming Code

The procedure used by the sender to encode the message encompasses the following steps −

- **Step 1** − Calculation of the number of redundant bits.
- **Step 2** − Positioning the redundant bits.
- **Step 3** − Calculating the values of each redundant bit.

Once the redundant bits are embedded within the message, this is sent to the user.

Step 1 − Calculation of the number of redundant bits.

If the message contains m number of data bits, r number of redundant bits are added to it so that mr is able to indicate at least $(m + r + 1)$ different states. Here, $(m + r)$ indicates location of an error in each of $(m + r)$ bit positions and one additional state indicates no error. Since, r bits can indicate 2^r states, 2^r must be at least equal to $(m + r + 1)$. Thus the following equation should hold $2^r \geq m+r+1$

Step 2 − Positioning the redundant bits.

The *r* redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc. They are referred in the rest of this text as r_1 (at position 1), r_2 (at position 2), r_3 (at position 4), r_4 (at position 8) and so on.

Step 3 − Calculating the values of each redundant bit.

The redundant bits are parity bits. A parity bit is an extra bit that makes the number of 1s either even or odd. The two types of parity are −

- **Even Parity** − Here the total number of bits in the message is made even.
- **Odd Parity** − Here the total number of bits in the message is made odd.

Each redundant bit, r_i, is calculated as the parity, generally even parity, based upon its bit position. It covers all bit positions whose binary representation includes a 1 in the i^{th} position except the position of r_i. Thus −

- r_1 is the parity bit for all data bits in positions whose binary representation includes a 1 in the least significant position excluding 1 (3, 5, 7, 9, 11 and so on)
- r_2 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 2 from right except 2 (3, 6, 7, 10, 11 and so on)
- r_3 is the parity bit for all data bits in positions whose binary representation includes a 1 in the position 3 from right except 4 (5-7, 12-15, 20-23 and so on)

Decoding a message in Hamming Code

Once the receiver gets an incoming message, it performs recalculations to detect errors and correct them. The steps for recalculation are −

- **Step 1** − Calculation of the number of redundant bits.
- **Step 2** − Positioning the redundant bits.
- **Step 3** − Parity checking.
- **Step 4** − Error detection and correction

Step 1 − Calculation of the number of redundant bits

Using the same formula as in encoding, the number of redundant bits are ascertained.

$2^r \geq m + r + 1$ where *m* is the number of data bits and *r* is the number of redundant bits.

Step 2 − Positioning the redundant bits

The *r* redundant bits placed at bit positions of powers of 2, i.e. 1, 2, 4, 8, 16 etc.

Step 3 – Parity checking

Parity bits are calculated based upon the data bits and the redundant bits using the same rule as during generation of c_1,c_2 ,c_3 ,c_4 etc. Thus

c_1 = parity(1, 3, 5, 7, 9, 11 and so on)

c_2 = parity(2, 3, 6, 7, 10, 11 and so on)

c_3 = parity(4-7, 12-15, 20-23 and so on)

Step 4 – Error detection and correction

The decimal equivalent of the parity bits binary values is calculated. If it is 0, there is no error.

Otherwise, the decimal value gives the bit position which has error. For example, if $c_1c_2c_3c_4$ = 1001, it implies that the data bit at position 9, decimal equivalent of 1001, has error. The bit is flipped to get the correct message.

Hamming Distance

Hamming distance is a metric for comparing two binary data strings. While comparing two binary strings of equal length, Hamming distance is the number of bit positions in which the two bits are different.

The Hamming distance between two strings, a and b is denoted as d(a,b).

It is used for error detection or error correction when data is transmitted over computer networks. It is also using in coding theory for comparing equal length data words.

Calculation of Hamming Distance

In order to calculate the Hamming distance between two strings, and , we perform their XOR operation, (a⊕ b), and then count the total number of 1s in the resultant string.

Example
Suppose there are two strings 1101 1001 and 1001 1101.

11011001 ⊕ 10011101 = 01000100. Since, this contains two 1s, the Hamming distance, d(11011001, 10011101) = 2.

Minimum Hamming Distance
In a set of strings of equal lengths, the minimum Hamming distance is the smallest Hamming distance between all possible pairs of strings in that set.

Example
Suppose there are four strings 010, 011, 101 and 111.

010 ⊕ 011 = 001, d(010, 011) = 1.

010 ⊕ 101 = 111, d(010, 101) = 3.

010 ⊕ 111 = 101, d(010, 111) = 2.

011 ⊕ 101 = 110, d(011, 101) = 2.

011 ⊕ 111 = 100, d(011, 111) = 1.

101 ⊕ 111 = 010, d(011, 111) = 1.

Hence, the Minimum Hamming Distance, dmin = 1.

parity check

In error-correcting codes, parity check has a simple way to detect errors along with a sophisticated mechanism to determine the corrupt bit location. Once the corrupt bit is located, its value is reverted (from 0 to 1 or 1 to 0) to get the original message.

How to Detect and Correct Errors?

To detect and correct the errors, additional bits are added to the data bits at the time of transmission.

- The additional bits are called **parity bits**. They allow detection or correction of the errors.
- The data bits along with the parity bits form a **code word**.

Parity Checking of Error Detection

It is the simplest technique for detecting and correcting errors. The MSB of an 8-bits word is used as the parity bit and the remaining 7 bits are used as data or message bits. The parity of 8-bits transmitted word can be either even parity or odd parity.

Even parity -- Even parity means the number of 1's in the given word including the parity bit should be even (2,4,6,....).

Odd parity -- Odd parity means the number of 1's in the given word including the parity bit should be odd (1,3,5,....).

Use of Parity Bit

The parity bit can be set to 0 and 1 depending on the type of the parity required.

•For even parity, this bit is set to 1 or 0 such that the no. of "1 bits" in the entire word is even. Shown in fig. (a).

•For odd parity, this bit is set to 1 or 0 such that the no. of "1 bits" in the entire word is odd. Shown in fig. (b).

Fig. (a)

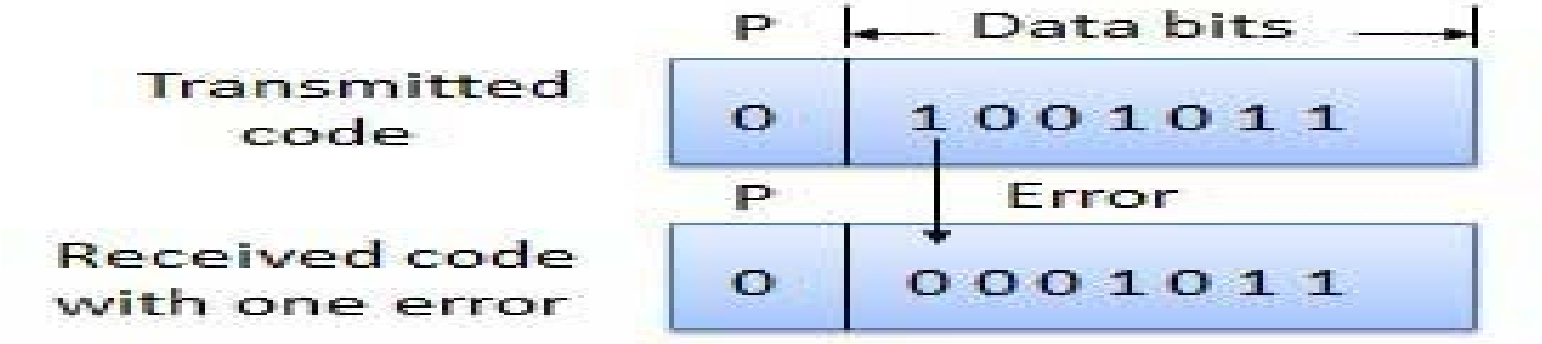
Fig. (b)

How Does Error Detection Take Place?

•Parity checking at the receiver can detect the presence of an error if the parity of the receiver signal is different from the expected parity.

•That means, if it is known that the parity of the transmitted signal is always going to be "even" and if the received signal has an odd parity, then the receiver can conclude that the received signal is not correct.

•If an error is detected, then the receiver will ignore the received byte and request for retransmission of the same byte to the transmitter.



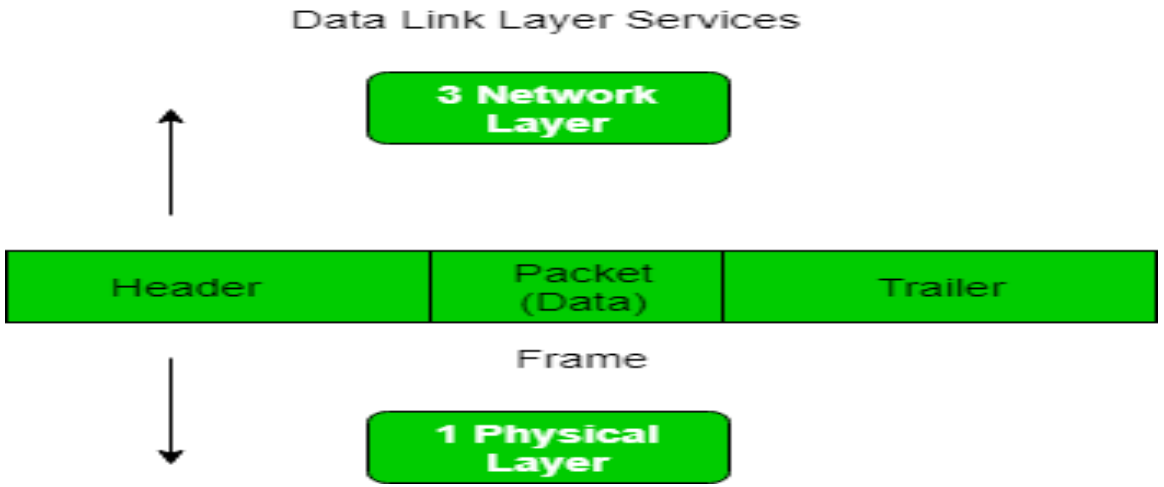
Framing in Data Link Layer

Framing in Data Link Layer

Frames are the units of digital transmission particularly in computer networks and telecommunications.

Frames are comparable to the packets of energy called photons in case of light energy.

Framing is a point-to-point connection between two computers or devices consists of a wire in which data is transmitted as a stream of bits.



Types of framing –

There are two types of framing:

1. Fixed size – The frame is of fixed size and there is no need to provide boundaries to the frame, length of the frame itself acts as delimiter.

Drawback: It suffers from internal fragmentation if data size is less than frame size

Solution: Padding

2. Variable size – In this there is need to define end of frame as well as beginning of next frame to distinguish. This can be done in two ways:

Length field – We can introduce a length field in the frame to indicate the length of the frame. Used in **Ethernet(802.3)**. The problem with this is that sometimes the length field might get corrupted.

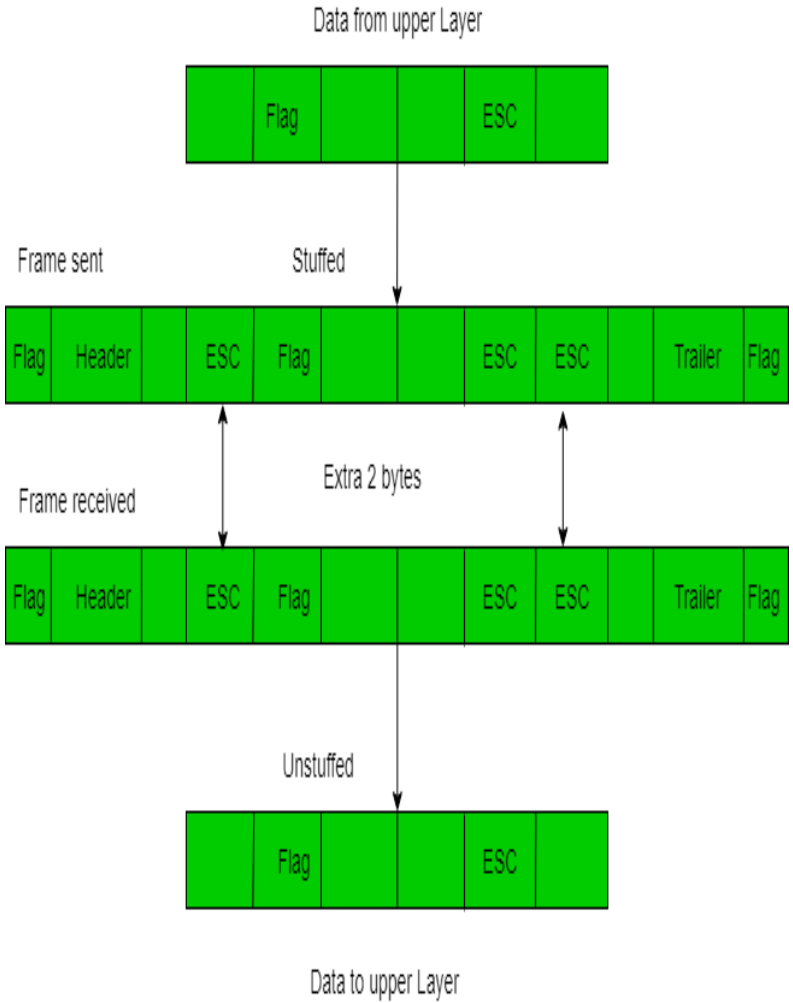
End Delimiter (ED) – We can introduce an ED(pattern) to indicate the end of the frame. Used in **Token Ring**.

1. **Character/Byte Stuffing:** If the pattern of the flag byte is present in the message byte, there should be a strategy so that the receiver does not consider the pattern as the end of the frame.

In character – oriented protocol, the mechanism adopted is byte stuffing.

In byte stuffing, a special byte called the escape character (ESC) is stuffed before every byte in the message with the same pattern as the flag byte.

If the ESC sequence is found in the message byte, then another ESC byte is stuffed before it.



2. Bit Stuffing: In bit-oriented framing, data is transmitted as a sequence of bits that can be interpreted in the upper layers both as text as well as multimedia data.

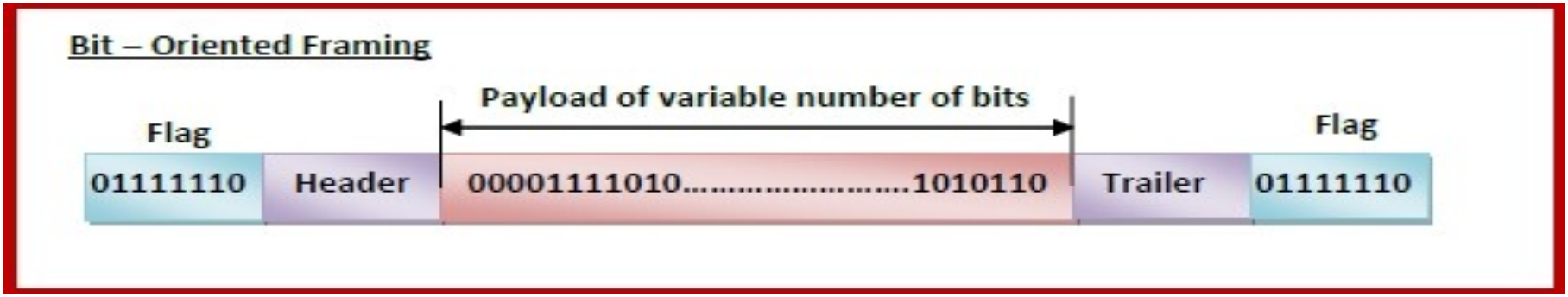
The parts of a frame in a character - oriented framing are −

Frame Header − It contains bits denoting the source and the destination addresses of the frame.

Payload field − It contains the message to be delivered. It is a variable sequence of bits.

Trailer − It contains the error detection and error correction bits.

Flags − Flags are a bit pattern that act as the frame delimiters signaling the start and end of the frame.
It is generally of 8-bits and comprises of six or more consecutive 1s. Most protocols use the 8-bit pattern 01111110 as flag.



Bit - oriented protocols are suited for transmitting any sequence of bits. So there are chances that the pattern of the flag bits is present in the message.

In order that the receiver does not consider this as end of frame, bit-stuffing mechanism is used.

Whenever a 0 bit is followed by five consecutive 1 bits in the message, an extra 0 bit is stuffed at the end of the five 1s.

When the receiver receives the message, it removes the stuffed 0s after each sequence of five 1s. The un-stuffed message is then sent to the upper layers.

Flow Control

Flow Control

When a data frame (Layer-2 data) is sent from one host to another over a single medium, it is required that the sender and receiver should work at the same speed. That is, sender sends at a speed on which the receiver can process and accept the data. What if the speed (hardware/software) of the sender or receiver differs? If sender is sending too fast the receiver may be overloaded, (swamped) and data may be lost.

Two types of mechanisms can be deployed to control the flow:

Noiseless Channels: simplest protocol, stop-and-wait

protocol.

Noisy Channel:-stop-and-wait Automatic Repeat Request (ARQ), go-back-n ARQ, Selective repeat ARQ, piggybacking.

Noiseless Channels: simplest protocol, stop-and-wait protocol.

Simplest Protocol

- In simplest protocol, there is no flow control and error control mechanism.

- It is a unidirectional protocol in which data frames travel in only one direction (from sender to receiver).

- Also, the receiver can immediately handle any received frame with a processing time that is small enough to be negligible.

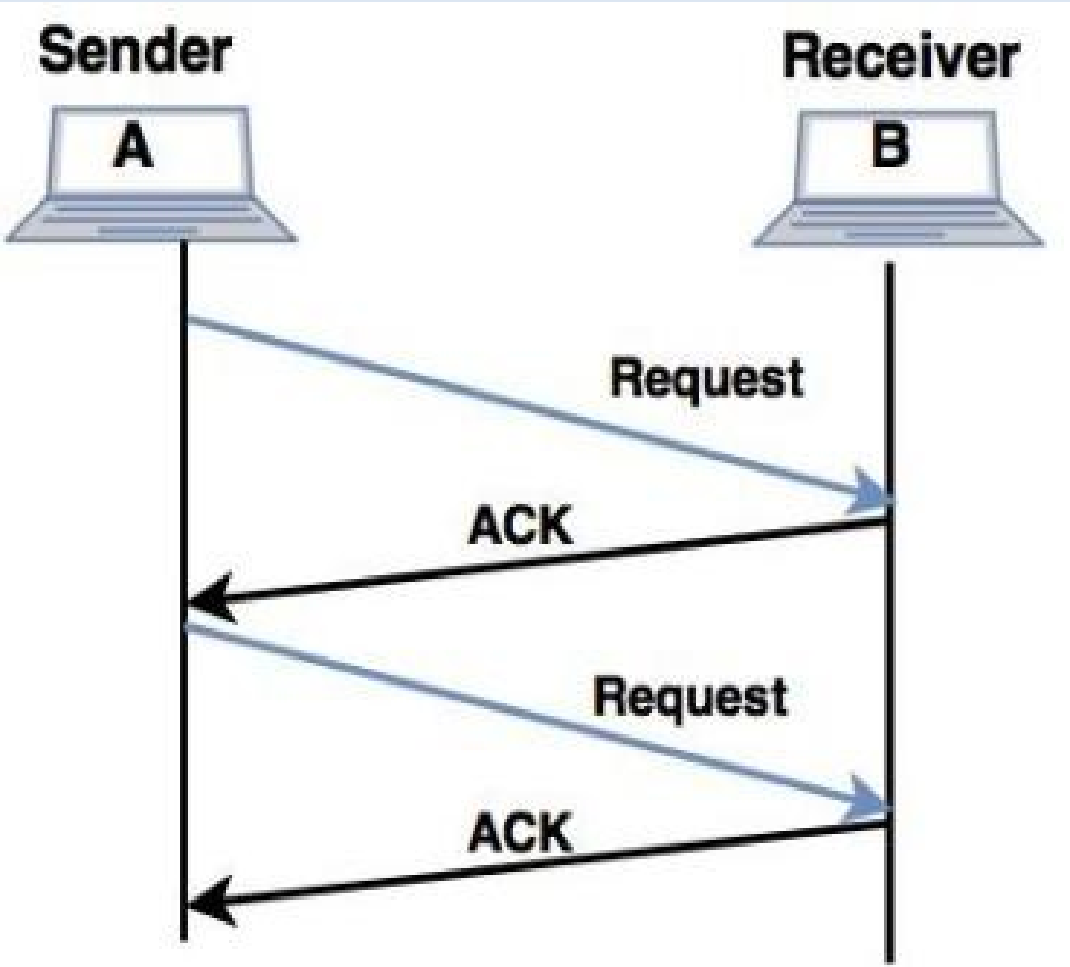
- The protocol consists of two distinct procedures :a sender and receiver.

- The sender runs in the data link layer of the source machine and the receiver runs in the data link layer of the destination machine.

- No sequence number or acknowledgements are used here.

Stop and Wait Protocol

- The simplest retransmission protocol is stop-and-wait.
- Transmitter (Station A) sends a frame over the communication line and then waits for a positive or negative acknowledgement from the receiver (station B).
- If no error occurs in the transmission, station B sends a positive acknowledgement (ACK) to station A.
- Now, the transmitter starts to send the next frame. If frame is received at station B with errors, then a negative acknowledgement(NAK) is sent to station A. In this case, station 'A' must retransmit the old packet in a new frame.
- There is also a possibility that the information frames or ACKs may get lost.
- Then, the sender is equipped with a timer. If no recognizable acknowledgement is received when the timer expires at the end of time out interval, the same frame is sent again.
- The sender which sends one frame and then waits for an acknowledgement before process is known as **stop and wait.**



Stop and Wait

Noisy Channels

Consider the normal situation of a communication channel that makes errors. Frames may be either damaged or lost completely.

Following are the protocols/methods.

Stop-and-wait ARQ

The following transition may occur in Stop-and-Wait ARQ:

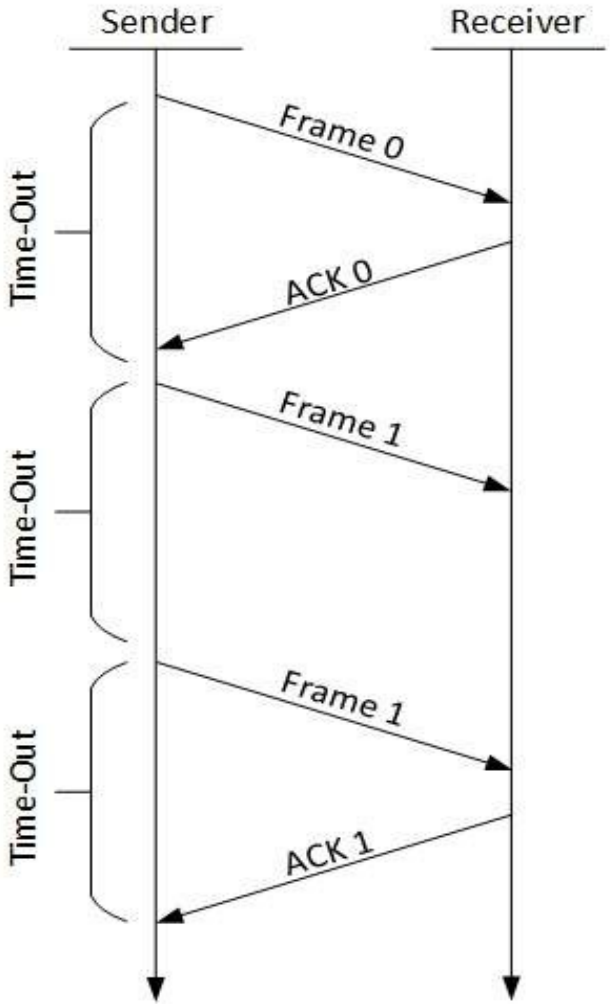
•The sender maintains a timeout counter.

•When a frame is sent, the sender starts the timeout counter.

•If acknowledgement of frame comes in time, the sender transmits the next frame in queue.

•If acknowledgement does not come in time, the sender assumes that either the frame or its acknowledgement is lost in transit. Sender retransmits the frame and starts the timeout counter.

•If a negative acknowledgement is received, the sender retransmits the frame.


Sender
Receiver
Frame 0
ACK 0
Frame 1
Frame 1
ACK 1
Time-Out
Time-Out
Time-Out

Go-Back-N ARQ

•Stop and wait ARQ mechanism does not utilize the resources at their best. When the acknowledgement is received, the sender sits idle and does nothing. In Go-Back-N ARQ method, both sender and receiver maintain a window.

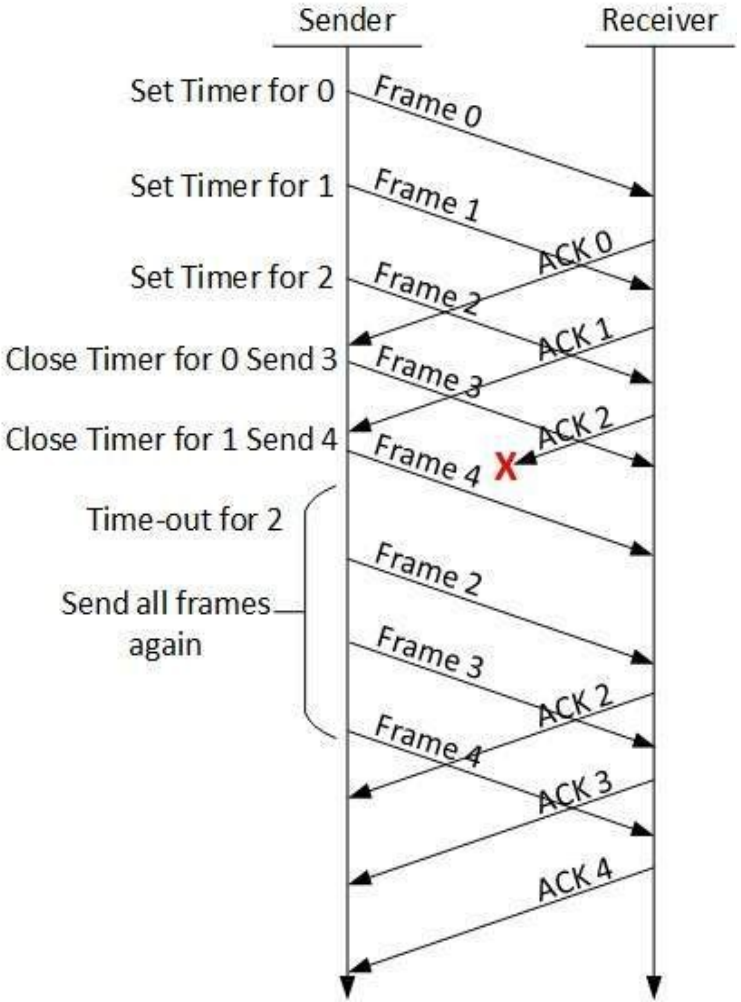
•The sending-window size enables the sender to send multiple frames without receiving the acknowledgement of the previous ones.

•The receiving-window enables the receiver to receive multiple frames and acknowledge them. The receiver keeps track of incoming frame's sequence number.

•When the sender sends all the frames in window, it checks up to what sequence number it has received positive acknowledgement.

•If all frames are positively acknowledged, the sender sends next set of frames.

•If sender finds that it has received NACK or has not receive any ACK for a particular frame, it retransmits all the frames after which it does not receive any positive ACK.


Sender
Receiver
Set Timer for 0
Frame 0
Set Timer for 1
Frame 1
ACK 0
Set Timer for 2
Frame 2
ACK 1
Close Timer for 0 Send 3
Frame 3
ACK 2
Close Timer for 1 Send 4
Frame 4
X
Time-out for 2
Send all frames again
Frame 2
Frame 3
ACK 2
Frame 4
ACK 3
ACK 4

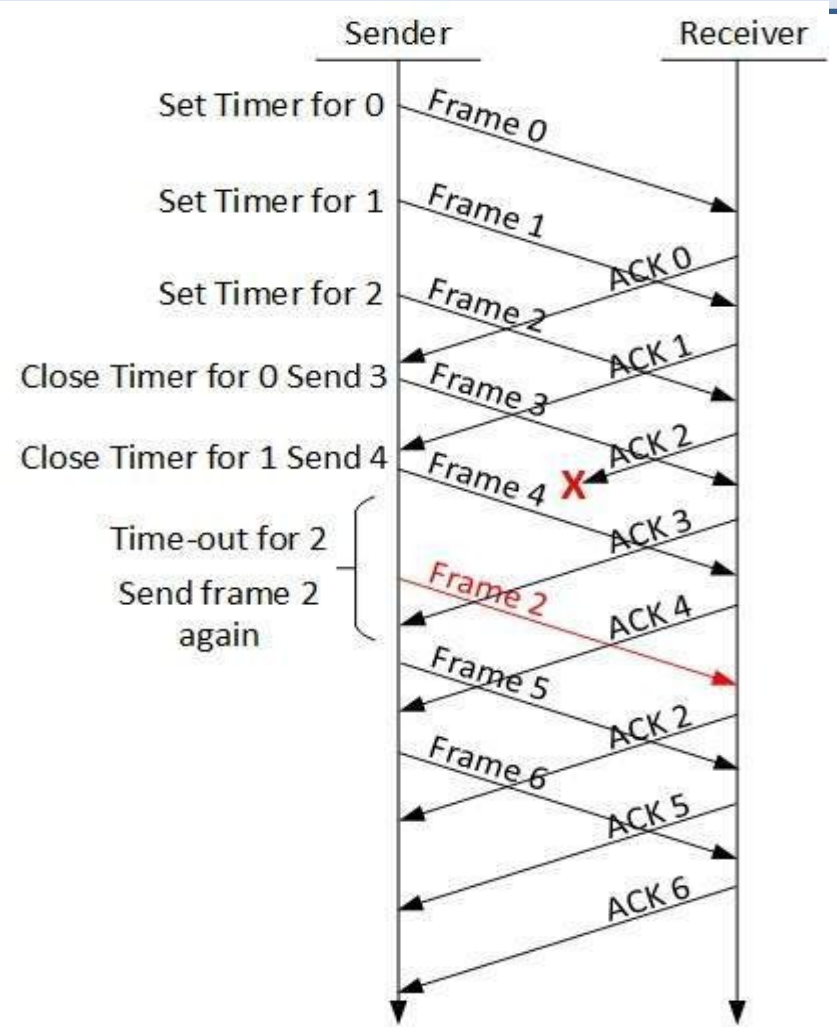
Selective Repeat ARQ

In Go-back-N ARQ, it is assumed that the receiver does not have any buffer space for its window size and has to process each frame as it comes.

This enforces the sender to retransmit all the frames which are not acknowledged.

In Selective-Repeat ARQ, the receiver while keeping track of sequence numbers, buffers the frames in memory and sends NACK for only frame which is missing or damaged.

The sender in this case, sends only packet for which NACK is received.


Sender
Receiver
Set Timer for 0
Frame 0
Set Timer for 1
Frame 1
ACK 0
Set Timer for 2
Frame 2
ACK 1
Close Timer for 0 Send 3
Frame 3
ACK 2
Close Timer for 1 Send 4
Frame 4
X
ACK 3
Time-out for 2
Send frame 2 again
Frame 2
ACK 4
Frame 5
ACK 2
Frame 6
ACK 5
ACK 6

piggybacking

The three principles governing piggybacking when the station X wants to communicate with station Y are −

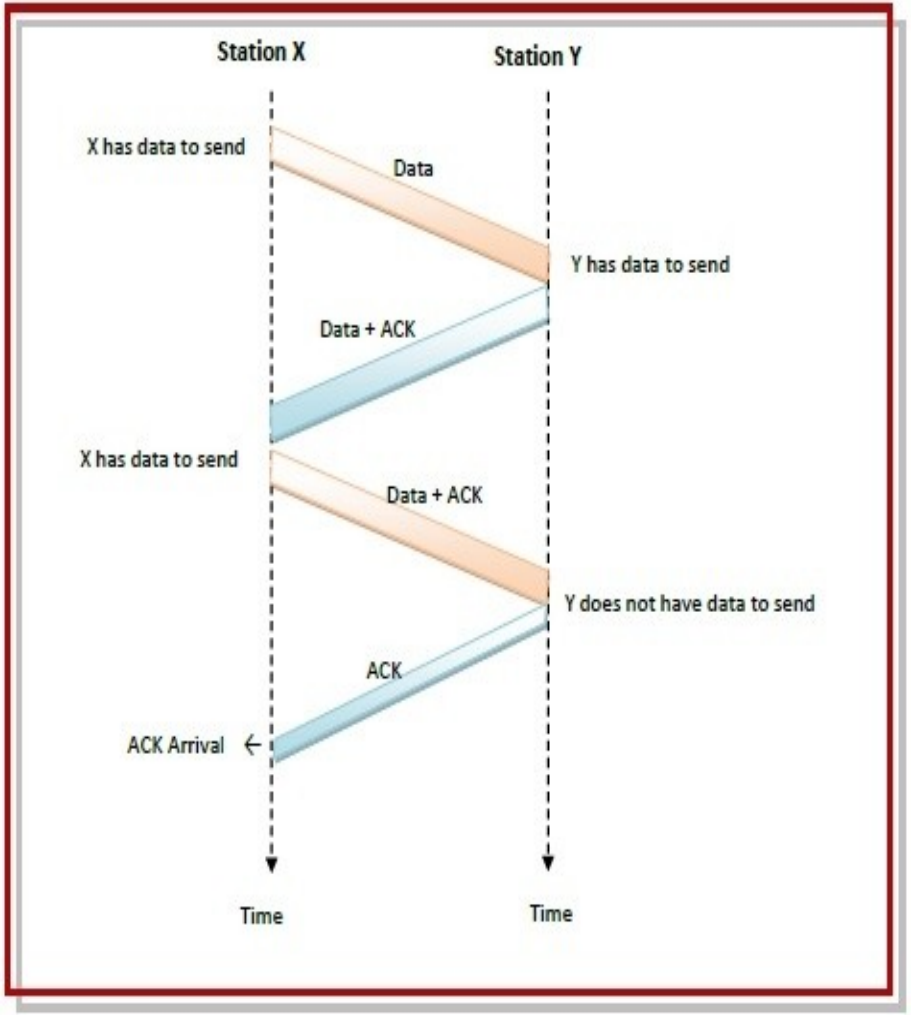
•If station X has both data and acknowledgment to send, it sends a data frame with the *ack* field containing the sequence number of the frame to be acknowledged.

•If station X has only an acknowledgment to send, it waits for a finite period of time to see whether a data frame is available to be sent.

•If a data frame becomes available, then it piggybacks the acknowledgment with it. Otherwise, it sends an ACK frame.

•If station X has only a data frame to send, it adds the last acknowledgment with it. The station Y discards all duplicate acknowledgments.

•Alternatively, station X may send the data frame with the *ack* field containing a bit combination denoting no acknowledgment.


Station X
Station Y
X has data to send
Data
Y has data to send
Data + ACK
X has data to send
Data + ACK
Y does not have data to send
ACK
ACK Arrival
Time
Time

Sender side

Given data = 7, 11, 12, 0, 6

Add 0 as Initial value of checksum.

data = 7, 11, 12, 0, 6, 0

Take sum of all = 36

36 cannot expressed in 4 bits. so extra 2 bits are wrapped and added with sum. Then complement the sum. The result is checksum.

eg>

```
  7
 11
 12
  0
  6
  0
 36
```

```
  1 0 0 1 0 0  = 36
+     1 0
-------------
  0 1 1 0      = 6
```

complement: 1001 = 9

Receiver side

```
  7
 11
 12
  0
  6
  9
 45
```

```
  1 0, 1 1 0 1  = 45
      1 0
-------------
  1 1 1 1       = 15
```

complement: 0000 = 0

No error.

Hamming code

This method is udes for error detection and correction (generally 1 bit)
7-bit are used in hamming codes

Rules (Considering 7 bit)
Data bits=4 bits
Parity bits=3
Now decide position of these seven bits i.e where we place data bits and parity bits
To solve this problem hamming use 2^n

i.E parity bit is of the power of 2 i.e (0,1,2,4,8,so on)

So in above seven bit code the position of parity are p1,p2,p4

So seven bits are represented as

D7	D6	D5	P4	D3	P2	P1

P1=D3,D5,D7
P2=D3,D6,D7
P4=D5,D6,D7

Example:- we want to send data 1011

So first place in there location as

D7	D6	D5	P4	D3	P2	P1
1	0	1		1		

Now we find values for parity bit,

P1=D3 D5 D7=111 -> 3 Once is odd, to make it even add 1 p1=1
P2=D3 D6 D7=101->2 Once is even, so p2=0
P4=D5 D6 D7=101 >2 Once is even, so p4=0

So frame 10

D7	D6	D5	P4	D3	P2	P1
1	0	1	0	1	0	1

NOW , we detect the errors

So frame 1010101 is transmitted by sender, so find error so in this frame we add errors at bit position 6.

So transmited frame becomes 1110101

D7	D6	D5	P4	D3	P2	P1
1	1	1	0	1	0	1

So considering even parity,

First check,

P1=1 and p1 is considered as P1=D3 D5 D7 1=111 SO IT IS EVEN NO ERROR

P2=0 and P2 Is considered as P2=D3 D6 D7 0=111 SO IT IS ODD ,ERROR

P4=0 and P4 Is considered as P4=D5 D6 D7 0=111 SO IT IS ODD ,ERROR

SO ERROR IS DETECTED IN THIS FRAME

So take frame with error 1110101 and assume even parity

D7	D6	D5	P4	D3	P2	P1
1	1	1	0	1	0	1

Now
P4 D5 D6 D7 =0 1 1 1 =3 once i.e odd so put p4=1. when contradiction shows change bit.

P2 D3 D6 D7=0 1 1 1 = 3 once i.e odd so put p2=1

P1 D3 D5 D7 = 1 1 1 1 = EVEN, Put p1=0

Convert bit into decimal p4 p2p1(110)2 = (6)

So at 6th position error occurs,